

Architecture Authoring Guide

Use this guide when you need to describe **system boundaries, parties, bounded contexts, services, and important dependencies**.

This guide is about the **big structural picture**: who owns what, what major parts exist, and how they relate.

What belongs here

- major systems or organizational parties
- bounded contexts or domain boundaries
- services or major components
- dependencies between major parts
- ownership and responsibility boundaries

What does not belong here

- detailed business process steps
- detailed screen behavior
- low-level infrastructure configuration

Core things to capture

- the main parties involved
- the contexts or major domains they own
- the services or major components that matter
- the purpose of each major part
- key dependencies and interfaces between parts

Core relationships to capture

- party -> context
- context -> service/component
- service -> dependency on another service/context

- owner -> owned structural element

Modus operandi

Start broad, then refine:

1. identify the big parties or systems
2. identify the main contexts they own
3. identify the important services/components inside those contexts
4. describe the most meaningful dependencies
5. stop before you fall into implementation detail

Prompt set

- what are the main system or business boundaries?
- who owns each major area?
- what major services/components exist?
- what depends on what?
- where are the most important interfaces or seams?

Free-text intake template

- **Architecture scope:**
- **Main parties/systems:**
- **Contexts/domains:**
- **Key services/components per context:**
- **Important dependencies:**
- **Ownership notes:**
- **Open structural concerns:**

Worked example

- HR Platform party owns Recruiting and Time Tracking contexts
- Recruiting context contains Candidate Service and Job Posting Service
- Candidate Service depends on Notification Service for status updates
- Reporting context consumes data from both Recruiting and Time Tracking

How this becomes YAML

A technical modeler or AI agent can transform this into architecture structures by mapping parties, contexts, services, and dependencies into a navigable structural

view.

Common mistakes

- listing every technical component instead of the meaningful structure
- mixing process flow with architecture boundaries
- naming dependencies without explaining why they exist
- omitting ownership